

Лабораторная работа №41

Разработка пользовательского интерфейса в ASP.NET Razor Pages

1 Цель работы

1.1 Научиться создавать веб-приложения, используя технологию ASP.NET Razor Pages.

2 Литература

2.1 <https://metanit.com/sharp/razorpages/> гл.2-3

3 Подготовка к работе

3.1 Повторить теоретический материал (см. п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

Создать проект «Веб-приложение ASP.NET Core (Майкрософт)».

5.1 Мастер-страница Layout (макет страницы)

Добавить в проект страницу `About.cshtml`, установить на ней заголовок через `ViewData["Title"] = "О нас"`.

Добавить в проект макет Razor (мастер-страницу) с названием `_MyLayout.cshtml` в папке `Pages/Shared`. Макет должен содержать:

- заголовок `<title>` с содержимым из `ViewData["Title"]`
- шапку сайта с меню (Главная, О нас – html-ссылки)
- основную секцию `@RenderBody()`
- подвал с годом и именем разработчика

Модифицировать исходный макет `_Layout.cshtml`, чтобы в нем была ссылка на страницу `About` (и последующие добавляемые по заданиям).

Требования:

- в корне `Pages` в файле `_ViewStart.cshtml` заменить `Layout` на `_MyLayout` по умолчанию.
- После тестирования – заменить на исходный макет `_Layout`.

5.2 Определение страниц, синтаксис Razor, ViewBag/ViewData

Добавить в проект страницу `SyntaxDemo.cshtml`. На ней применить следующие конструкции Razor:

- вывод переменной (`@имя`), например, для текущей даты или случайного целого числа,
- условный блок (`@if/else`), например для времени суток (утро, день, вечер, ночь),
- цикл (`@for` или `@foreach`), например для вывода таблицы умножения или календаря на неделю,
- блок кода (`@{ ... }`) для объединения нескольких команд C# на странице,
- вывод HTML без кодирования (`@Html.Raw()`), для того, чтобы вывести значение строковой переменной, содержащей html-тэги.

Передать на страницу через `ViewBag` список из 3 имён пользователей и вывести их маркированным списком.

Через `ViewData` передать текущую дату и вывести её.

Требования:

- использовать **только `ViewBag` и `ViewData`** (без свойств модели),
- в `OnGet()` заполнить `ViewBag` и `ViewData`,
- применить комментарии в коде Razor (`@* ... *@`).

5.3 Tag-хелперы, создание ссылок, контекст страницы

Добавить в проект страницу `LinksDemo.cshtml`.

На `LinksDemo.cshtml` разместить следующие ссылки с использованием **tag-хелперов**:

1. на страницу `Index` (корневую)
2. на страницу `About`
3. на страницу `Privacy` с параметром `?id=5`
4. на текущую страницу с якорем `#top`

На странице вывести информацию из **контекста страницы**:

- путь к текущему файлу (`this.GetType().Assembly.Location` или через `Path`)
- IP-адрес клиента (`HttpContext.Connection.RemoteIpAddress`)
- тип запроса (`Request.Method`)

Требования:

- использовать `<a asp-page="...">` и `asp-route-*` для параметров.
- контекст получать в модели страницы и передавать через свойства (не `ViewBag`).

5.4 Формы с tag-хелперами и статусные коды

Добавить в проект страницу `FormWithStatus.cshtml` с формой для обратной связи (поля: Имя, Email, Сообщение).

Использовать **tag-хелперы форм**:

- `<form asp-page-handler="Send">`
- `<input asp-for="Name">`, `<input asp-for="Email">`, `<textarea asp-for="Message">`
- `<span asp-validation-for="Name">` (без подключения jQuery — просто заглушка)

При отправке формы:

- если все поля заполнены, вернуть **статусный код 201 (Created)** с текстом «Сообщение отправлено»,
- если имя или сообщение короче 3 символов, вернуть **400 (BadRequest)**.

Требования:

- использовать `[BindProperty]` для полей `Name`, `Email`, `Message`.
- вернуть результат через `StatusCode(int, string)`.
- не использовать перенаправления: статус должен отобразиться на той же странице.

5.5 Полный CRUD без БД

Добавить в проект страницу `ToDoList.cshtml` для списка задач (хранить в статическом списке в памяти).

Функции:

- отображение всех задач (использовать `@foreach` с Razor)
- добавление новой задачи через форму с tag-хелпером `<input asp-for="NewTask">`
- удаление задачи по ID через отдельную форму с обработчиком `OnPostDelete`

Требования:

- во `_ViewImports.cshtml` добавить `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers`.
- при удалении использовать ссылку-кнопку: `<button asp-page-handler="Delete" asp-route-id="...">`
- в подвале Layout вывести количество запросов к текущей странице (использовать контекст – счетчик в сессии или статическую переменную).

Требования к статусным кодам:

Если пользователь пытается удалить несуществующую задачу, вернуть **404 (NotFound)**.

6 Порядок выполнения работы

- 6.1 Запустить MS Visual Studio и создать на C# набор требуемых проектов.
- 6.2 Выполнить все задания из п.5. При выполнении заданий использовать минимально возможное количество команд и переменных и выполнять форматирование и рефакторинг кода.
- 6.3 Ответить на контрольные вопросы.

7 Содержание отчета

- 7.1 Титульный лист
- 7.2 Цель работы
- 7.3 Ответы на контрольные вопросы
- 7.4 Вывод

8 Контрольные вопросы

- 8.1 Какой синтаксис используется для встраивания C# в Razor?
- 8.2 Для чего служит файл `_ViewImports.cshtml`?
- 8.3 Как подключить мастер-страницу через `_ViewStart.cshtml`?
- 8.4 Как создать ссылку с помощью tag-хелпера `asp-page`?
- 8.5 Какой атрибут tag-хелпера привязывает поле к свойству модели?